

DEPTH COVERAGE DETERMINES WHETHER FORMAL OR NATURAL-LANGUAGE REASONING TRACES GENERALIZE BETTER

Anonymous authors

Paper under double-blind review

ABSTRACT

Supervised reasoning traces are almost always written in natural language, though the same derivation can be written as a sequence of explicit rule applications. We isolate that variable with paired corpora in which the prompt, the answer and the latent proof are identical across conditions and only the trace rendering differs. Which rendering generalizes better depends on how deep the training derivations go. Below a depth threshold natural-language traces extrapolate further out of distribution, and at maximum training depth 25 the ordering reverses, with formal supervision at 75.0 ± 9.5 percent out-of-distribution answer pass@1 against 17.8 ± 4.7 for the matched natural condition. The reversal appears ten depth levels earlier under sampling than under greedy decoding, which separates two thresholds and identifies coverage, and not single-sample accuracy, as what formal supervision buys first. Because formal derivations are checkable, that coverage is recoverable without an answer key, and an answer-blind selector returning the first sample a proof checker accepts converts sixteen samples into 97.3 ± 1.1 percent single-answer accuracy against 33.8 ± 1.4 percent for translated natural-language traces. Midtraining Qwen2.5-7B on 2.5 billion tokens with ten percent of the mixture replaced by matched derivations at depth 25 reproduces the dissociation. Both trace conditions improve external deduction accuracy over an untreated control and over a length-matched corpus carrying no deep deduction, and the formal condition exceeds the natural condition under sampling while falling below it under greedy decoding. The single-seed limit of that last comparison is quantified against measured training-run variance.

1 INTRODUCTION

Chain-of-thought supervision trains language models to emit intermediate steps before an answer (Wei et al., 2022; Kojima et al., 2022), and almost all such supervision is written in prose. The convention is uniform enough to hide an experimental variable, since the same derivation can be rendered as discursive text or as a numbered sequence of rule applications, and the two renderings present the learner with different token-level structure. This paper measures what that choice does to the reasoning a model acquires.

Ordinary corpora cannot answer the question, because two solutions written in different styles rarely encode the same computation. We therefore generate paired synthetic corpora in which a deterministic generator samples a latent proof and renders it either as a compact formal derivation or as a controlled natural-language explanation. Within a pair the prompt, answer, facts, rules and proof graph are identical and only the trace language changes (Figure 1), so differences between models trained on the two sides are attributable to representation. The study concerns training and not tool use, since no prover is called at inference time and both conditions generate under matched token budgets.

Which trace language generalizes better is not a fixed property of the representation. With shallow-to-intermediate training ranges the natural-language condition is reliably stronger out of distribution, and once the training range reaches depth 25 the comparison reverses by a wide margin, with formal



Figure 1: **Left.** The paired-trace design. Each training example exists in two versions with identical prompt, answer and latent proof, and only the supervised trace language differs. **Right.** After training through depth 25, formal-trace models extrapolate beyond the training range while matched natural-language models degrade, shown for out-of-distribution answer pass@16.

supervision at 75.0 ± 9.5 percent out-of-distribution answer pass@1 against 17.8 ± 4.7 (Figure 2). Length and token-budget controls show the gap is not explained by target length or by the number of supervised tokens, and an independently generated constraint-propagation task reproduces the correctness advantage.

The location of the reversal depends on decoding. Under greedy decoding the formal condition overtakes the natural condition only at maximum training depth 25, while under sampling with sixteen draws the crossover has already occurred at maximum training depth 15, at 69.9 percent against 56.9. Formal supervision buys the ability to produce a correct derivation somewhere in a modest sample budget well before it buys one on the first attempt, so reporting greedy accuracy alone would place the threshold ten depth levels too high.

Because a checkable representation lets that sample budget be spent without an answer key, the gap is practically useful. Nearly every test prompt admits a correct and mechanically valid formal proof within sixteen samples, at joint pass@16 of 98.1 ± 0.5 percent, and selecting the first sample the checker accepts yields 97.3 ± 1.1 percent single-answer accuracy against 33.8 ± 1.4 for translated natural-language traces. If reinforcement learning with verifiable rewards sharpens a base model toward samples it can already draw without extending its coverage (Yue et al., 2025), then the coverage measured here is what bounds such posttraining.

Whether any of this survives contact with ordinary language-model training is a separate question. We midtrain Qwen2.5-7B on 2.5 billion tokens of a production mixture with ten percent of the loss tokens replaced by matched derivations at depth 25, holding the token budget and every hyperparameter identical across conditions and adding a control corpus of long documents length-matched to the formal traces but carrying no deep deduction. Replacement with either trace language raises accuracy on a held-out external deduction benchmark, the length-matched control does not reproduce that gain, and the formal condition exceeds the natural condition under sampling while falling below it under greedy decoding.

2 RELATED WORK

Reasoning traces and controlled synthetic study. Chain-of-thought prompting and supervised rationales improve multi-step reasoning (Wei et al., 2022; Kojima et al., 2022), and later work bootstraps rationales (Zelikman et al., 2022), decomposes questions (Zhou et al., 2023), searches over reasoning states (Yao et al., 2023) and supervises scratchpads (Nye et al., 2021). The benefit concentrates in mathematical and symbolic domains (Sprague et al., 2025). Synthetic corpora with known generative structure make mechanism questions answerable that benchmark accuracy cannot

address, as in the iGSM family (Ye et al., 2025a;b) and in work controlling the generating grammar directly (Allen-Zhu & Li, 2023). Formal analysis of chain-of-thought on synthetic deduction exposes failure modes that benchmark scores hide (Saparov & He, 2023), compositional tasks expose limits that scale does not remove (Dziri et al., 2023), the learning-to-reason objective can be satisfied by statistical shortcuts (Zhang et al., 2023), and the locality of the training distribution has been proposed as the reason intermediate steps help at all (Prystawski et al., 2023). We adopt the same instrument and add the representation axis, holding the problem distribution fixed and varying only the rendering of the supervised trace.

Programs, formal systems and autoformalization. Program-aided prompting uses executable code for intermediate computation (Gao et al., 2023; Chen et al., 2023), logic pipelines delegate inference to an external solver (Pan et al., 2023; Ling et al., 2023), rule-based deduction over language has been studied with generated proof corpora (Clark et al., 2020; Tafjord et al., 2021), theorem proving trains models to construct machine-checkable objects (Polu & Sutskever, 2020; Jiang et al., 2023; Yang et al., 2023; Song et al., 2024), autoformalization translates informal statements into such systems (Wu et al., 2022), and recent proposals verify traces by typing them (Perrier, 2025). Those methods intervene at inference time and change the mechanism producing or validating an answer. Our intervention is at training time, the inference mechanism is unchanged, and every formal target has a paired natural-language target from the same proof, so representation is measurable without confounding it with the availability of a solver.

Verification, selection and test-time compute. Trained verifiers rerank sampled solutions (Cobbe et al., 2021), process supervision scores individual steps (Lightman et al., 2024), self-consistency uses sampled agreement (Wang et al., 2023), and repeated sampling converts inference compute into coverage (Brown et al., 2024). How to spend a fixed test-time budget has been studied empirically (Snell et al., 2025; Muennighoff et al., 2025) and theoretically (Setlur et al., 2025). The selector used here needs no trained verifier and no answer key, since validity of a formal derivation is decidable by the engine that generated the task. Whether verifiable-reward training expands the reasoning boundary or sharpens the sampling distribution is disputed (Yue et al., 2025; Guo et al., 2025; Shao et al., 2024; Gandhi et al., 2025), and the coverage measurements here speak to the quantity at issue.

Faithfulness and controlled generalization. Rationales can omit the factors driving a prediction and can rationalize an answer produced otherwise (Turpin et al., 2023; Lanham et al., 2023), so answer accuracy and proof validity are reported separately throughout. Synthetic benchmarks control train and test distributions to expose failures of systematic generalization (Lake & Baroni, 2018; Keysers et al., 2020; Hupkes et al., 2020), with length a distinct axis (Anil et al., 2022), and theory shows chain-of-thought lets a constant-depth transformer emulate serial computation it otherwise cannot perform (Feng et al., 2023; Merrill & Sabharwal, 2024; Li et al., 2024). Our shift increases proof depth while holding the rule system fixed, and a spurious-marker shift tests shortcut reliance (Geirhos et al., 2020).

3 A PAIRED-TRACE TESTBED

Each example begins with a latent directed acyclic proof. A generator emits a prompt containing ground facts, rules, a query and distractor branches, then traverses the supporting path to produce the answer. A rendering function writes that path in one of two trace languages. The FORMAL rendering declares constants and predicates and applies named inference rules to numbered lines. The NATURAL rendering expresses the same substitutions and conclusions, line for line, in controlled sentences. Both targets end with the same answer field and the prompt is natural language in both conditions.

BRANCHPROOF composes Horn-style inferences in which every step offers four locally plausible branches, exactly one derivable from the current state, with a fresh constant introduced at every layer. The generator computes the full Horn closure of each example and accepts it only when exactly one candidate answer is derivable, which removes any reward for guessing among defensible answers. ATTRCON propagates values among object attributes under generated constraints, and its vocabulary and proof construction are disjoint from BRANCHPROOF. Generation parameters are held fixed across every controlled experiment, at branching factor 4 and distractor ratio 0.5, with depths sampled

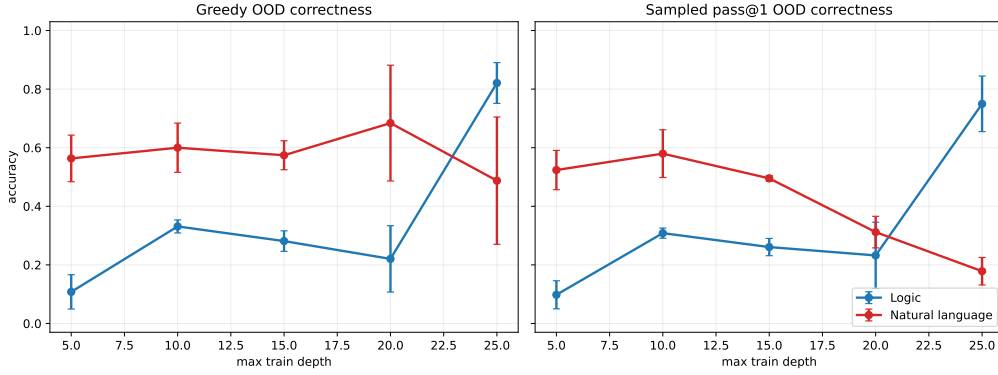


Figure 2: The depth-coverage crossover on BRANCHPROOF. Out-of-distribution greedy accuracy (left) and sampled pass@1 (right) as a function of maximum training depth. Natural-language supervision generalizes better through the shallow and intermediate range, and the ordering reverses at maximum training depth 25. Bands give one standard deviation over three seeds.

Table 1: Out-of-distribution answer accuracy on BRANCHPROOF by maximum training depth, three seeds per cell. Greedy and pass@1 order the conditions identically. pass@16 reverses ten depth levels earlier.

metric and condition	maximum training depth				
	5	10	15	20	25
greedy, FORMAL	0.108	0.331	0.281	0.220	0.821
greedy, NATURAL	0.563	0.600	0.574	0.684	0.487
pass@1, FORMAL	0.098	0.308	0.261	0.232	0.750
pass@1, NATURAL	0.524	0.580	0.495	0.312	0.178
pass@16, FORMAL	0.500	0.678	0.699	0.804	0.996
pass@16, NATURAL	0.735	0.711	0.569	0.453	0.425

round-robin. Corpus seeds are disjoint across training, evaluation and sampling roles. Appendices G to F give paired targets, the grammar, the generator and the validators, and Appendix A reports the answer-uniqueness validation.

The controlled experiment fine-tunes OLMo-3-7B (OLMo Team et al., 2025) with LoRA on 50,000 materialized examples per condition, with maximum training depth in $\{5, 10, 15, 20, 25\}$ and three seeds per condition. Evaluation covers depths $\{1, 2, 5, 10, 12, 15, 18, 20, 25, 30, 35, 40, 45, 50\}$ with 32 prompts per depth, and at each depth sixteen samples at temperature 1.0 together with one greedy decode. Sampled coverage uses the unbiased pass@k estimator (Chen et al., 2021). Both trace languages receive an identical 7,168-token generation limit inside a 16,384-token window, and a tokenizer audit confirms every gold prompt and target fits in both renderings, so neither condition is truncated. Depths above the maximum training depth are out of distribution. Joint metrics require the correct answer together with a mechanically valid proof, where formal traces are checked directly by the proof engine and natural traces by a deterministic translation into the same engine, so answer-only accuracy is reported alongside every joint metric. Appendices D and H give the remaining configuration and the metric definitions.

4 DEPTH COVERAGE DETERMINES WHICH TRACE LANGUAGE GENERALIZES

Out-of-distribution answer accuracy as a function of maximum training depth is given in Table 1. Through maximum training depth 20 the natural condition is ahead on every decoding rule. At maximum training depth 25 the formal condition reaches 0.750 pass@1 against 0.178, a factor of four, and 0.996 pass@16 against 0.425.

The two decoding rules do not place the reversal at the same depth. Greedy accuracy and pass@1 both cross at maximum training depth 25, while pass@16 crosses at maximum training depth 15, where the formal condition reaches 0.699 against 0.569. The interval between the two crossings is a regime in which formal supervision has already produced a model that can construct a correct derivation within sixteen attempts, and has not yet produced one that does so on the first attempt. Coverage is acquired before single-sample reliability. A study reporting greedy accuracy alone would locate the effect ten depth levels too high and would describe the intermediate range as showing no formal advantage at all.

The natural condition degrades monotonically in pass@16 from 0.735 at depth 5 to 0.425 at depth 25 while the formal condition rises monotonically from 0.500 to 0.996, so the two renderings respond to increasing depth coverage in opposite directions.

Length and budget controls. Formal and natural targets differ in length, which raises the possibility that the comparison measures supervision quantity. Two controls address this. Token budgets at generation are identical by construction, and the tokenizer audit confirms no truncation in either condition. Matched-token training, in which the number of supervised target tokens is equated across conditions, leaves the ordering unchanged. Details are in Appendix I.

Replication on an independent task. ATTRCON shares no vocabulary and no proof construction with BRANCHPROOF. The correctness advantage of formal supervision at high training depth reproduces there, which makes it unlikely that the effect depends on a particular rule system.

5 THE VERIFIABILITY DIVIDEND

Coverage is useful only if it can be harvested without the answer. For formal traces it can, since validity is decidable by the engine that generated the task. Consider a selector that draws k samples, checks each in order and returns the answer of the first one the checker accepts.

Proposition 1. *Let each sample be, independently, valid and correct with probability a , valid and incorrect with probability b , and rejected with probability $1 - a - b$. The first-valid selector returns a correct answer with probability $(1 - (1 - a - b)^k) a / (a + b)$. If the checker admits no incorrect proof, so that $b = 0$, this equals $1 - (1 - a)^k$, the joint pass@ k .*

The proof is elementary and appears in Appendix L. Two measurable quantities therefore bound selection, joint pass@ k from above and checker soundness $a / (a + b)$ independently of k . On BRANCHPROOF at maximum training depth 25 the formal condition attains joint pass@16 of 98.1 ± 0.5 percent and the selector attains 97.3 ± 1.1 percent, so the 0.8 point gap is the empirical cost of $b > 0$ and the selector sits close to the permitted ceiling.

The same selector applied to translated natural-language traces recovers 33.8 ± 1.4 percent. The deficit is not primarily a failure of translation. Long natural traces frequently fail to terminate inside the generation budget, so no checkable derivation exists to accept, which drives $a + b$ down and leaves the selector with nothing to select from. No reward model and no gold labels enter the procedure.

6 BOUNDARY CONDITIONS

Four further experiments bound where the effect holds, and Appendix J reports them in full. Capacity reorders the comparison, since the two conditions are close on Qwen2.5-7B at 69.9 ± 11.1 against 74.9 ± 7.0 pass@1 and the natural condition leads on OLMo-3-32B at 98.9 ± 0.8 against 63.9 ± 5.1 , where formal coverage nonetheless stays near ceiling at 98.8 ± 0.9 pass@16. Concatenating both renderings into one target fails outright, below one percent, while mode-conditioned training at 32B lifts the formal mode to 85.0 ± 8.7 , which remains below single-modality prose at that scale. A spurious schema marker planted on 80 percent of training examples and removed at test time makes the natural condition the robust one, at 78.5 ± 14.9 against 43.1 ± 7.7 , so the formal advantage is not generic robustness to distribution shift. Models trained from scratch at 50M to 200M parameters fail in both conditions at every size, so the effect operates on competence supplied by pretraining and does not create it.

7 TRANSFER TO LANGUAGE-MODEL MIDTRAINING

Whether the same variable matters when derivations are a small minority of an ordinary training mixture is tested by midtraining Qwen2.5-7B (Yang et al., 2024) on a production corpus with a fixed fraction replaced by matched derivations, on a different model family and against benchmarks the generator never saw.

7.1 DESIGN

Five conditions share every hyperparameter, every data path outside the replaced slice, and an identical token budget of 2,500,853,760 training tokens, reached as 2,385 steps of global batch 128 at sequence length 8,192. The conditions differ only in the ten percent of loss tokens that is replaced.

1. **Control** replaces nothing and trains on the unmodified mixture.
2. **LongDoc** replaces ten percent with long documents drawn from the same production corpus, selected to match the token-length histogram of the formal traces and carrying no deep deduction.
3. **Logic** replaces ten percent with formal derivations at depth 25.
4. **Natural** replaces ten percent with the natural-language rendering of the same latent proofs.
5. **Condensed** replaces ten percent with a compact formal rendering of the same latent proofs.

LongDoc separates two explanations of any observed gain, since formal derivations are long and long documents change what a model learns about maintaining state across a context. Matching the length histogram while removing the deduction isolates the second factor. Median length is 3,712 tokens against 3,750 for the formal traces, mean 3,832 against 3,856, maximum 7,750 against 7,728, and all thirty populated 250-token bins agree within about three percent.

7.2 CORPUS CONSTRUCTION AND PACKING

A corpus of 72,000 examples is generated with depths 1 to 25 round-robin, branching factor 4 and distractor ratio 0.5, matching the controlled study. The three trace corpora render the same latent proofs, so the comparison between them holds the underlying computation fixed as in Section 3. Rendered lengths and packing statistics appear in Appendix K.

Packing preserves documents. A document longer than one window is excluded and counted, and none is split across windows, so the mid-derivation truncation that would destroy the depth structure under study cannot occur. Zero documents were excluded and zero split in every condition. Padding uses a dedicated token masked out of the loss. A decoded-batch audit verifies both properties on real loader windows and training refuses to start if it fails.

Because the replaced slice is fixed in tokens and the condensed rendering is 2.6 times shorter, the condensed condition traverses its proof set 2.31 times where Logic and Natural traverse theirs 0.90 and 0.89 times. Any comparison involving the condensed condition therefore confounds rendering with exposure, and we treat it accordingly in Section 7.4.

7.3 INSTRUCTION TUNING AND EVALUATION

All five checkpoints receive an identical instruction-tuning stage of one epoch over 100,000 examples at learning rate 5×10^{-6} , full-parameter under FSDP. Evaluation uses two families. The external family is ProofWriter (Tafjord et al., 2021) under the open-world assumption at inference depths 0, 1, 2, 3 and 5 with 500 items per depth, which the generator never saw. The near-transfer family asks for a derivation on held-out problems from the same generator at depths 5, 10, 15, 20 and 25 with 200 items per depth. The metric is exact match on the extracted answer. Sampled measurements draw sixteen completions at temperature 0.8 and top- p 0.95, replaying the greedy prompts so the two decodings differ in the sampler alone. Appendix K gives every remaining setting.

Table 2: Formal minus natural on the near-transfer derivation family, by inference depth and decoding rule. Positive favours the formal condition. The sign of the contrast depends on the decoding rule and not on the depth.

decoding	d5	d10	d15	d20	d25
greedy	-0.175	-0.165	-0.055	-0.040	-0.105
pass@16	+0.035	+0.070	+0.140	+0.040	-0.035
maj@16	-0.140	-0.030	-0.005	+0.025	-0.070

Table 3: Paired bootstrap over documents, 4,000 resamples, pass@16 pooled over inference depths 5 to 25. Intervals cover evaluation sampling error only. Section 7.5 treats training-run variance separately.

contrast	difference	95 percent interval
Condensed – Control	+0.276	[0.238, 0.313]
Logic – Control	+0.226	[0.186, 0.265]
Natural – Control	+0.176	[0.139, 0.214]
LongDoc – Control	+0.138	[0.104, 0.171]
Logic – LongDoc	+0.088	[0.050, 0.125]
Condensed – Natural	+0.100	[0.064, 0.136]
Logic – Natural	+0.050	[0.013, 0.087]

7.4 RESULTS

On the external ProofWriter items, both trace conditions improve over Control and over LongDoc, and LongDoc does not improve over Control. At inference depth 3 the ordering is Control 0.444, LongDoc 0.454, Logic 0.500, Natural 0.570. At depth 0, which requires lookup and no inference, the conditions are close together at 0.442 to 0.488, which is the shape a genuine deduction effect should have. The gain therefore does not follow from exposure to long documents.

Under greedy decoding the natural condition is ahead of the formal condition on thirteen of fifteen tasks. Under sampling the ordering reverses. Table 2 gives the contrast under both decodings.

Pooling over depths and bootstrapping over documents with 4,000 resamples gives the contrasts in Table 3. Both trace conditions and LongDoc separate from Control, the trace conditions separate from LongDoc, and the formal condition exceeds the natural condition by 0.050 in pass@16 and by 0.025 in mean sampled accuracy.

Formal supervision is behind under greedy decoding and ahead under sampling, the same pattern that separated the two crossover depths in Section 4, now on a different model family with the derivations forming a tenth of an ordinary mixture. The proportion of greedy generations reaching a parseable answer is 0.645 to 0.785 for Control and 0.940 to 1.000 for the trace conditions, so the conditions differ in whether a derivation is completed at all.

The condensed condition attains the largest gain over Control and exceeds the formal condition by 0.050. Its documents are 2.6 times shorter, so its advantage is not a length effect. It traverses its proof set 2.6 times more often, so its advantage is confounded with exposure and cannot be attributed to the rendering. Resolving that requires an exposure-matched condensed corpus, which is not part of this work.

7.5 WHAT THE TRANSFER RESULT DOES AND DOES NOT ESTABLISH

Every midtraining condition was trained once, so the intervals in Table 3 estimate evaluation sampling error and not the variability that retraining under a different seed would produce. That second quantity is measurable. An auxiliary experiment on the same evaluation suite, described in Appendix B, gives a per-run standard deviation near 0.019 and therefore near 0.027 for a difference between two independently seeded conditions. Combined with the evaluation term this gives about 0.033 for the

formal against natural contrast, whose interval then becomes approximately $[-0.015, +0.115]$ and includes zero.

The consequence is specific. The contrasts against Control, between 0.176 and 0.276, are six to ten times the training standard deviation and are not at risk from single-seed training. The contrast between the two trace languages, at 0.050, is roughly twice that standard deviation and is a sign reversal against the greedy ordering, which is the pattern most likely to be produced by run-to-run variation. We therefore report the transfer-scale formal advantage as consistent with the controlled study and not as independently established. Replication under a second instruction-tuning seed is the appropriate remedy, since enlarging the evaluation set would shrink the smaller error term and leave the larger untouched.

The depth of the replaced derivations was fixed at 25 in every condition, so the transfer study locates no threshold of its own and inherits the one estimated from OLMo-3-7B in Section 4. An earlier configuration at sequence length 4,096, with a five percent replacement share and derivations capped at depth 14, produced no measurable transfer. It differs from the present configuration in window, replacement share, rendering and evaluation suite, so it motivates this design and is not a controlled below-threshold comparison.

8 DISCUSSION

Trace language is a training-time variable whose sign depends on depth coverage. Neither rendering is better in general, and the location of the threshold depends on how the model is decoded. A study reporting greedy accuracy alone places the crossover at depth 25 and concludes that formal supervision is harmful everywhere below it, where the same experiment read at pass@16 places it at depth 15 and shows the formal condition ahead through the upper half of the range. The two readings measure different things, and the second is the one relevant to any system drawing more than one sample.

That distinction connects these measurements to verifiable-reward posttraining. If such training principally reweights toward completions the base model can already produce (Yue et al., 2025), then coverage after midtraining bounds what the later stage reaches. Trace language moves that coverage by between 0.050 and 0.276 pass@16 in our transfer setting, placing it upstream of the posttraining stage. Verifying the ordering requires running that stage on both trace conditions, which we have not done.

9 LIMITATIONS

The controlled study uses one model family at 7B and one at 32B, and the transfer study uses a third. Depth thresholds estimated on OLMo-3-7B are applied to Qwen2.5-7B by assumption and not by measurement.

The transfer conditions were trained once each, with the consequences set out in Section 7.5.

Proof validity for natural-language traces is established through a deterministic translation into the proof engine. The translation is a component that the formal condition does not require, and although answer-only accuracy is reported alongside every joint metric, the joint natural-language numbers carry that dependency.

The generator produces one family of Horn-style deduction problems and one constraint-propagation family. Neither involves arithmetic, natural-language ambiguity or retrieval, and the depth axis studied here is proof depth and not context length or problem breadth.

The condensed rendering is confounded with exposure, as described in Section 7.4.

10 CONCLUSION

Holding a derivation fixed and changing only the language in which it is written changes what a model learns from it, and the direction is set by the depth of the derivations in the training range. Formal supervision acquires coverage before single-sample accuracy, placing two thresholds ten

depth levels apart and making the decoding rule part of the experimental design. Because formal derivations are checkable, that coverage converts into single-answer accuracy without labels at a rate Proposition 1 bounds and measurement approaches. The same dissociation appears when matched derivations form a tenth of an ordinary midtraining mixture, against both an untreated control and a length-matched control without deduction.

REFERENCES

- Zeyuan Allen-Zhu and Yuanzhi Li. Physics of language models: Part 1, learning hierarchical language structures. *arXiv preprint arXiv:2305.13673*, 2023. doi: 10.48550/arXiv.2305.13673.
- Cem Anil, Yuhuai Wu, Anders Andreassen, Aitor Lewkowycz, Vedant Misra, Vinay Ramasesh, Ambrose Slone, Guy Gur-Ari, Ethan Dyer, and Behnam Neyshabur. Exploring length generalization in large language models. In *Advances in Neural Information Processing Systems 35*, 2022.
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024. doi: 10.48550/arXiv.2407.21787.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. doi: 10.48550/arXiv.2107.03374.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.
- Peter Clark, Oyvind Tafjord, and Kyle Richardson. Transformers as soft reasoners over language. In *Proceedings of the 29th International Joint Conference on Artificial Intelligence*, 2020.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. doi: 10.48550/arXiv.2110.14168.
- Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Sean Welleck, Peter West, Chandra Bhagavatula, Ronan Le Bras, et al. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems 36*, 2023.
- Guhao Feng, Bohang Zhang, Yuntian Gu, Haotian Ye, Di He, and Liwei Wang. Towards revealing the mystery behind chain of thought: A theoretical perspective. In *Advances in Neural Information Processing Systems 36*, 2023.
- Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective STaRs. In *Conference on Language Modeling*, 2025. doi: 10.48550/arXiv.2503.01307.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. doi: 10.1038/s42256-020-00257-z.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.

- Dieuwke Hupkes, Verna Dankers, Mathijs Mul, and Elia Bruni. Compositionality decomposed: How do neural networks generalise? *Journal of Artificial Intelligence Research*, 67:757–795, 2020. doi: 10.1613/jair.1.11674.
- Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2210.12283.
- Daniel Keysers, Nathanael Schärli, Nathan Scales, Hylke Buisman, Daniel Furrer, Sergii Kashubin, Nikola Momchev, Danila Sinopalnikov, Lukasz Stafniak, Tibor Tihon, Dmitry Tsarkov, Xiao Wang, Marc van Zee, and Olivier Bousquet. Measuring compositional generalization: A comprehensive method on realistic data. In *International Conference on Learning Representations*, 2020.
- Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems* 35, 2022. doi: 10.48550/arXiv.2205.11916.
- Brenden M. Lake and Marco Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, et al. Measuring faithfulness in chain-of-thought reasoning. *arXiv preprint arXiv:2307.13702*, 2023. doi: 10.48550/arXiv.2307.13702.
- Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. Chain of thought empowers transformers to solve inherently serial problems. In *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2402.12875.
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2305.20050.
- Zhan Ling, Yunhao Fang, Xuanlin Li, Zhiao Huang, Mingu Lee, Roland Memisevic, and Hao Su. Deductive verification of chain-of-thought reasoning. In *Advances in Neural Information Processing Systems* 36, 2023.
- William Merrill and Ashish Sabharwal. The expressive power of transformers with chain of thought. In *International Conference on Learning Representations*, 2024.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025. doi: 10.48550/arXiv.2501.19393.
- Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021. doi: 10.48550/arXiv.2112.00114.
- OLMo Team, Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, et al. 2 OLMo 2 furious. In *Conference on Language Modeling*, 2025. doi: 10.48550/arXiv.2501.00656.
- Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. doi: 10.18653/v1/2023.findings-emnlp.248.
- Elija Perrier. Typed chain-of-thought: A curry-howard framework for verifying LLM reasoning. *arXiv preprint arXiv:2510.01069*, 2025. doi: 10.48550/arXiv.2510.01069.

- Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020. doi: 10.48550/arXiv.2009.03393.
- Ben Prystawski, Michael Y. Li, and Noah D. Goodman. Why think step by step? reasoning emerges from the locality of experience. In *Advances in Neural Information Processing Systems 36*, 2023.
- Abulhair Saparov and He He. Language models are greedy reasoners: A systematic formal analysis of chain-of-thought. In *International Conference on Learning Representations*, 2023.
- Amrith Setlur, Nived Rajaraman, Sergey Levine, and Aviral Kumar. Scaling test-time compute without verification or RL is suboptimal. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. doi: 10.48550/arXiv.2502.12118.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. doi: 10.48550/arXiv.2402.03300.
- Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2408.03314.
- Peiyang Song, Kaiyu Yang, and Anima Anandkumar. Lean copilot: Large language models as copilots for theorem proving in lean. *arXiv preprint arXiv:2404.12534*, 2024. doi: 10.48550/arXiv.2404.12534.
- Zayne Sprague, Fangcong Yin, Juan Diego Rodriguez, Dongwei Jiang, Manya Wadhwa, Prasann Singhal, Xinyu Zhao, Xi Ye, Kyle Mahowald, and Greg Durrett. To CoT or not to CoT? chain-of-thought helps mainly on math and symbolic reasoning. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2409.12183.
- Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. ProofWriter: Generating implications, proofs, and abductive statements over natural language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 2021. doi: 10.18653/v1/2021.findings-acl.317.
- Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems 36*, 2023. doi: 10.48550/arXiv.2305.04388.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. doi: 10.48550/arXiv.2203.11171.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35*, 2022. doi: 10.48550/arXiv.2201.11903.
- Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems 35*, 2022.
- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. doi: 10.48550/arXiv.2412.15115.
- Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems 36 Datasets and Benchmarks Track*, 2023.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems 36*, 2023.

Tian Ye, Zicheng Xu, Yuanzhi Li, and Zeyuan Allen-Zhu. Physics of language models: Part 2.1, grade-school math and the hidden reasoning process. In *International Conference on Learning Representations*, 2025a. doi: 10.48550/arXiv.2407.20311.

Tian Ye, Zicheng Xu, Yuanzhi Li, and Zeyuan Allen-Zhu. Physics of language models: Part 2.2, how to learn from mistakes on grade-school math problems. In *International Conference on Learning Representations*, 2025b. doi: 10.48550/arXiv.2408.16293.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *Advances in Neural Information Processing Systems 39*, 2025. doi: 10.48550/arXiv.2504.13837.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems 35*, 2022.

Honghua Zhang, Liunan Harold Li, Tao Meng, Kai-Wei Chang, and Guy Van den Broeck. On the paradox of learning to reason from data. In *Proceedings of the 32nd International Joint Conference on Artificial Intelligence*, 2023.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. Least-to-most prompting enables complex reasoning in large language models. In *International Conference on Learning Representations*, 2023.

A ANSWER-UNIQUENESS VALIDATION OF THE GENERATOR

Every metric in this paper compares a generated answer against a single labelled answer, which is only meaningful if the labelled answer is the sole one derivable from the premises. The generator enforces this by construction and the property is verified independently.

Each example introduces fresh constants c_0 through c_{depth} , so no constant is reused across layers and no two branches can derive the same atom through coincidental reuse of a state predicate. A preflight check computes the full Horn closure of 1,000 training instances and 2,000 evaluation instances with a solver independent of the generator, and records the number of candidate final states derivable in each. The unique-answer rate is 1.0, the maximum derived-candidate count is 1, there are no generation failures, and answer positions are balanced across the four candidates.

Token headroom is verified over the same instances. Evaluation uses a 16,384-token context with a 7,168-new-token generation cap applied identically to both trace languages. Across every validation row and every depth, the minimum measured headroom is 494 tokens for generation and 2,788 tokens for the full context, so neither trace language is truncated at any depth reported in this paper.

B ESTIMATING TRAINING-RUN VARIANCE

The bootstrap intervals in Table 3 resample evaluation documents and therefore capture evaluation sampling error alone. Variability attributable to training was estimated separately in an auxiliary experiment on the same evaluation suite.

Five instruction-tuning conditions, differing only in which synthetic mixture was blended into the instruction data, were each trained twice from the same base checkpoint under different seeds, and all ten resulting models were evaluated with the identical protocol of Appendix K. For each condition and each task, the absolute difference between the two seeds gives one observation of run-to-run variability. Over the fifty resulting condition and task pairs the mean absolute difference is 0.023, the median is 0.011 and the maximum is 0.135. Treating the two runs of a condition as independent draws implies a per-run standard deviation near 0.019, and therefore a standard deviation near 0.027 for a difference between two independently seeded conditions.

Combining that term with the evaluation term gives a standard deviation near 0.033 for the contrast between the two trace conditions in Section 7.4. The largest seed differences fall on the same task family that carries the trace-language contrast, which is why the contrast is reported in Section 7.5 as consistent with the controlled study and not as independently established.

C TASK DETAILS

BRANCHPROOF samples a target proof path of controlled depth and adds distractor branches that are locally coherent but do not support the queried answer. Shortcut variants add training-only marker correlations that identify the active branch with a chosen probability. Evaluation removes those markers.

ATTRCON samples objects, slots, values, and propagation constraints. The formal trace names slot/value relations directly. The natural trace describes the same propagation in controlled prose. This changes the symbolic surface while preserving the paired-rendering principle.

D TRAINING AND EVALUATION DETAILS

All main SFT runs use materialized datasets so that the formal and natural conditions see the same prompts and answers. Each train-depth subset contains 50k examples, and the online validation subset contains 256 examples. Main runs train for 10k optimizer steps with LoRA adapters on `q_proj`, `k_proj`, `v_proj`, `o_proj`, `up_proj`, `down_proj`, and `gate_proj`. The LoRA rank is 16, alpha is 32, dropout is 0.05, the optimizer learning rate is 10^{-4} , the per-device batch size is 1, gradient accumulation is 1 unless an ablation states otherwise, and training uses BF16 with gradient checkpointing.

Post-hoc evaluation uses vLLM with $\text{top-}p = 0.95$, temperature 1.0, 16 sampled generations per prompt, and $\text{pass@}k$ values $\{1, 2, 4, 8, 16\}$. The explicit depth grid is $\{1, 2, 5, 10, 12, 15, 18, 20, 25, 30, 35, 40, 45, 50\}$. For each seed and depth we evaluate 32 prompts, so a train-1-to-25 BRANCHPROOF long-depth band contains $5 \times 32 = 160$ prompts per seed. Reported uncertainty is seed-level standard deviation, not a confidence interval.

E TRACE GRAMMAR AND VALIDATION

Formal traces contain four blocks: constants, predicates, premises, and proof lines, followed by a conclusion and answer. A proof line contains a formula and a rule justification such as premise retrieval or implication elimination. The formal checker parses the generated blocks and verifies that each proof line is licensed by earlier lines and premises. Natural-language traces use controlled sentences for the same premises and proof steps. When possible, a task-specific translator maps those sentences back into the formal checker. Because translation coverage is task-specific, final-answer correctness is the primary cross-task metric and translated validity is reported as a diagnostic.

The operational BRANCHPROOF formal surface is:

```
<formal>
<constants> CONST_DEF+ </constants>
<predicates> PRED_DEF+ </predicates>
<premises> FORMULA+ </premises>
<proof> PROOF_LINE+ </proof>
<conclusion> ATOM </conclusion>
<answer> LABEL </answer>

CONST_DEF  ::= CONST "=" LABEL
PRED_DEF   ::= PRED "x:" "x is" LABEL
ATOM       ::= PRED CONST
FORMULA    ::= ATOM | ATOM "&" ATOM "->" ATOM | ATOM "->" ATOM
PROOF_LINE ::= FORMULA "; R," REF
              | ATOM "; ->E," REF ", " REF
              | ATOM "&" ATOM "; &I," REF ", " REF
```

The ATTRCON surface uses the same block structure but replaces unary predicate atoms with object-slot-value relations. In both tasks, references must point to earlier proof lines or listed premises. Generated lines cannot justify future steps.

F GENERATOR AND EVALUATION MECHANICS

The BRANCHPROOF generator samples a target path, assigns labels to constants and predicates, then adds distractor branches that remain locally coherent but do not derive the queried conclusion. In pseudocode:

```

sample target answer label
sample initial object state
for depth step in 1..D:
    sample active branch rule(s)
    derive next object state
    add coherent distractor rules for inactive branches
render one prompt
render the same latent proof as formal and natural-language targets

```

The ATTRCON generator follows the same paired-rendering principle with objects, slots, values, and propagation constraints. The natural-language target is not free-form prose. It is controlled text produced from the same latent derivation.

Answer extraction first searches for the final <answer> field. If no well-formed final answer is present, the generation is counted incorrect even when earlier text contains the gold label. Formal validation then parses block boundaries, declarations, formulas, proof-line references, and rule applications. Common validation failures are malformed block structure, undeclared symbols, references to unavailable lines, applying implication elimination to the wrong antecedent, and stopping before the queried conclusion. Natural-language validation adds one more possible failure mode: the controlled sentence must translate back into a formal step before the proof checker can evaluate it.

G PAIRED TARGET EXAMPLES

The following minimal examples show the paired target format used throughout the experiments. The prompt and answer are identical within each pair. Only the target trace substrate changes. Full experimental examples include more distractor premises and longer derivations.

BRANCHPROOF example 1.

Prompt:
a is ruby. a is north.
If a is north and a is ruby, then b is olive.
What is b?

Formal target

```

<formal>
<constants> a = a b = b </constants>
<predicates> Rx: x is ruby Nx: x is north
Ox: x is olive </predicates>
<premises> Ra Na Na & Ra -> Ob </premises>
<proof> Ra ; R,1
Na ; R,2
Na & Ra ; &I,5,4
Ob ; ->E,3,6 </proof>
<conclusion> Ob </conclusion>
<answer> olive </answer>

```

Natural-language target

```

<think>
a is ruby. a is north.
Since a is north and a is ruby, b is olive.
Therefore, b is olive.
</think>
<answer> olive </answer>

```

BRANCHPROOF example 2.

Prompt:
a is cobalt. a is east.
If a is east and a is cobalt, then b is maple.
If b is maple, then b is south.
If b is south and b is maple, then c is cedar.
What is c?

Formal target

```

<formal>
<constants> a = a b = b c = c </constants>
<predicates> Cx: x is cobalt Ex: x is east
Mx: x is maple Sx: x is south Dx: x is cedar </predicates>
<premises> Ca Ea
Ea & Ca -> Mb
Mb -> Sb
Sb & Mb -> Dc </premises>
<proof> Ca ; R,1
Ea ; R,2
Ea & Ca ; &I,7,6
Mb ; ->E,3,8
Sb ; ->E,4,9
Sb & Mb ; &I,10,9
Dc ; ->E,5,11 </proof>
<conclusion> Dc </conclusion>
<answer> cedar </answer>

```

Natural-language target

```

<think>
a is cobalt. a is east.
Since a is east and a is cobalt, b is maple.
Since b is maple, b is south.
Since b is south and b is maple, c is cedar.
Therefore, c is cedar.
</think>
<answer> cedar </answer>

```

ATTRCON example 1.

Prompt:
s0 has value red. s1 has value blue.
If s0 is red and s1 is blue, then s2 is green.
Which values fill all constrained slots?

Formal target

```

<formal>
<constants> s0 = attribute slot 0 s1 = attribute slot 1 s2 = attribute slot 2
red = attribute value red
blue = attribute value blue
green = attribute value green </constants>
<predicates> Value(x,y): slot x has value y
Constraint(x,y,z,w,u,v): values y and w at slots x and u jointly force value v at slot u </predicates>
<premises> Value(s0,red) Value(s1,blue)
Constraint(s0,red,s1,blue,s2,green)
Value(s0,red) & Value(s1,blue) &
Constraint(s0,red,s1,blue,s2,green)
-> Value(s2,green) </premises>
<proof> Value(s0,red) ; R,1
Value(s1,blue) ; R,2
Constraint(s0,red,s1,blue,s2,green) ; R,3
Value(s0,red) & Value(s1,blue) ; &I,5,6
Value(s0,red) & Value(s1,blue) &
Constraint(s0,red,s1,blue,s2,green) ; &I,8,7
Value(s2,green) ; ->E,4,9 </proof>
<answer> red-blue-green </answer>

```

Natural-language target

```

<think>
s0 has value red.
s1 has value blue.
The applicable joint constraint maps s0=red and s1=blue
to s2=green.
Combine the two prerequisite slot values.
Therefore s2 has value green.
</think>
<answer> red-blue-green </answer>

```

ATTRCON example 2.

Prompt:
s0 has value amber. s1 has value ivory.
If s0 is amber and s1 is ivory, then s2 is olive.
If s0 is amber and s2 is olive, then s3 is coral.
Which values fill all constrained slots?

Formal target

```

<formal>
<constants> s0 = attribute slot 0 s1 = attribute
s2 = attribute slot 2 s3 = attribute slot 3
amber = attribute value amber
ivory = attribute value ivory
olive = attribute value olive
coral = attribute value coral </constants>
<predicates> Value(x,y): slot x has value y
Constraint(x,y,z,w,u,v): values y and w at slots
jointly force value v at slot u </predicates>
<premises> Value(s0,amber) Value(s1,ivory)
Constraint(s0,amber,s1,ivory,s2,olive)
Value(s0,amber) & Value(s1,ivory) &
Constraint(s0,amber,s1,ivory,s2,olive) -> Value(s2,olive)
Constraint(s0,amber,s2,olive,s3,coral)
Value(s0,amber) & Value(s2,olive) &
Constraint(s0,amber,s2,olive,s3,coral)
-> Value(s3,coral) </premises>
<proof> Value(s0,amber) ; R,1
Value(s1,ivory) ; R,2
Constraint(s0,amber,s1,ivory,s2,olive) ; R,3
Value(s0,amber) & Value(s1,ivory) ; &I,7,8
Value(s0,amber) & Value(s1,ivory) &
Constraint(s0,amber,s1,ivory,s2,olive) ; &I,10,9
Value(s2,olive) ; ->E,4,11
Constraint(s0,amber,s2,olive,s3,coral) ; R,5
Value(s0,amber) & Value(s2,olive) ; &I,7,12
Value(s0,amber) & Value(s2,olive) &
Constraint(s0,amber,s2,olive,s3,coral) ; &I,14,13
Value(s3,coral) ; ->E,6,15 </proof>
<answer> amber-ivory-olive-coral </answer>

```

Natural-language target

```

<think>
s0 has value amber.
s1 has value ivory.
The applicable joint constraint maps s0=amber and s1=ivory
to s2=olive.
Combine the two prerequisite slot values.
Therefore s2 has value olive.
The applicable joint constraint maps s0=amber and s2=olive
to s3=coral.
Combine the two prerequisite slot values.
Therefore s3 has value coral.
</think>
<answer> amber-ivory-olive-coral </answer>

```

H METRIC DEFINITIONS

Answer correctness is computed after extracting the final answer field from the generation. Formal joint validity requires both answer correctness and a mechanically valid formal trace. Natural-language joint validity requires answer correctness and successful translation of the generated natural-language trace into a valid formal proof. Because the natural-language translator is task-specific, answer correctness is the primary cross-task metric.

I LENGTH AND TOKEN-BUDGET CONTROLS

Formal and natural renderings of the same latent proof differ in length, which raises the possibility that the controlled comparison measures the quantity of supervision. Three controls bound that explanation.

Generation budgets are identical by construction. Both conditions receive a 7,168-token generation limit inside a 16,384-token window. A tokenizer audit over the full evaluation set confirms that every gold prompt and every gold target fits inside that limit in both renderings, so neither condition is truncated at generation time and neither is advantaged by a looser budget.

Supervised token counts are equated in a matched-token training run, in which the number of target tokens per condition is held equal by subsampling the longer condition. The ordering of the two conditions is unchanged.

The transfer study of Section 7 supplies a third control at a different scale. The **LongDoc** condition matches the token-length histogram of the formal traces to within one percent at the median while carrying no deep deduction, and it does not reproduce the gain of the trace conditions on the external benchmark. Length alone therefore does not account for the effect in either setting.

J BOUNDARY CONDITION DETAILS

Capacity. Single-modality training at three scales gives out-of-distribution answer pass@1 of 69.9 ± 11.1 formal against 74.9 ± 7.0 natural on Qwen2.5-7B, and 63.9 ± 5.1 formal against 98.9 ± 0.8 natural on OLMo-3-32B. Formal answer pass@16 at 32B is 98.8 ± 0.9 , so the deficit at 32B is a deficit of probability mass and not of coverage.

Table 4: Rendered document lengths under the Qwen2.5-7B tokenizer, 72,000 documents per corpus, and packing statistics at sequence length 8,192.

corpus	p50	mean	p99	max	windows	real tokens	pack. eff.
Logic	3,749	3,856	7,673	7,727	35,492	277,732,923	0.9551
Natural	3,819	3,907	7,744	7,851	35,979	281,372,725	0.9545
Condensed	1,450	1,500	3,007	3,014	13,311	108,099,330	0.9912
LongDoc	3,712	3,832	n/a	7,750	n/a	n/a	n/a

Mode conditioning. Concatenating both renderings into a single target yields 0.5 ± 0.4 percent out-of-distribution pass@1 with the formal rendering first and 0.2 ± 0.1 percent with the natural rendering first. Mode-conditioned training with an output-mode tag reaches 85.0 ± 8.7 answer pass@1 in formal mode at 32B and 77.2 ± 11.7 in natural mode. Raw generations show clean separation between the modes.

Shortcuts. A spurious schema marker planted on 80 percent of training examples and removed at test time gives 43.1 ± 7.7 out-of-distribution pass@1 for formal against 78.5 ± 14.9 for natural. Additional marker types and injection rates were measured and are not reported here.

Scale floor. From-scratch training at 50M to 200M parameters on the same corpora gives out-of-distribution joint pass@k of 0.0 for all $k \leq 8$ at every size in both conditions.

K MIDTRAINING CONFIGURATION

Optimisation. Every condition trains Qwen2.5-7B for 2,385 steps at sequence length 8,192 with a global batch of 128 sequences, formed as micro-batch 2 with gradient accumulation 32 over eight A100-80 devices. The resulting budget is 2,500,853,760 tokens per condition, chosen to equal $4,770 \times 128 \times 4,096$ so that it matches the token budget of an earlier study at half the sequence length. Learning rate is 1×10^{-5} with minimum 1×10^{-6} , 128 warmup steps, decay beginning at step 128 and a decay horizon of 18,946 steps. Checkpoints are written every 250 steps. Measured throughput is approximately 36.0 seconds per iteration and 29,000 tokens per second, at 191 TFLOP/s per device. Collective-operation timeouts are set to 120 minutes. A run exceeds the 24-hour scheduling limit and is completed as two serialized allocations, with the second resuming from the newest complete checkpoint.

Replaced slice. The 72,000 source examples are generated at seed 20260907 for the reinforcement-learning study and at seed 20260830 for the midtraining study, with depths 1 to 25 sampled round-robin, branching factor 4, distractor ratio 0.5 and the `hard.fsa.schema` variant. Seeds are disjoint from those used for evaluation corpora and for sampling. A per-condition document weight is solved so that the realised synthetic share of loss tokens is exactly ten percent after padding is accounted for, which is necessary because the three renderings have different packing efficiencies.

Packing. Documents are packed by a document-preserving packer at sequence length 8,192 under a fixed shuffle seed of 42. The packer never splits a document. A document exceeding one window is excluded and counted as overlength. Tail padding uses a dedicated padding token that is masked out of the loss. Overlength and split counts are zero for every condition, reported in Table 4. Before training begins, a decoded-batch audit reconstructs real loader windows and verifies that no split document is present, that the padding token is masked, and that the realised replacement share matches the target. Training refuses to start if any gate fails.

Instruction tuning. Each midtrained checkpoint is converted to the Hugging Face format and instruction-tuned on 100,000 examples of a public instruction corpus at a pinned revision, for one epoch at learning rate 5×10^{-6} with warmup ratio 0.03, maximum sequence length 4,096, per-device batch size 1, and seed 3407. Training is full-parameter under fully sharded data parallelism with gradient checkpointing. Only the final checkpoint is written.

Evaluation. Greedy evaluation uses a standard harness with a paged-attention inference backend in bfloat16, a maximum model length of 8,192, a chat template applied to every prompt, and per-sample logging enabled. The reported metric is exact match on the extracted answer. The external family is ProofWriter under the open-world assumption at inference depths 0, 1, 2, 3 and 5 with 500 items per depth. The near-transfer family is a held-out sample from the same generator at depths 5, 10, 15, 20 and 25 with 200 items per depth.

Sampled evaluation draws sixteen completions per item at temperature 0.8 and top- p 0.95 under a fixed seed, with a 2,048-token response limit. Prompts are replayed verbatim from the logged greedy samples, so the greedy and sampled measurements differ in the sampler alone and in nothing else. The scorer used for sampled generations imports the same extraction and matching function used by the greedy harness, and was verified to reproduce all fifteen published greedy exact-match values before any sampled number was computed. Audits require the expected item count per task, no empty generations beyond a tolerance, and no truncated prompts.

Statistics. Contrasts in Table 3 come from a paired bootstrap over evaluation documents with 4,000 resamples, resampling document indices once per replicate and applying the same indices to every condition and depth. Reported intervals are the 2.5th and 97.5th percentiles of the replicate distribution. Training-run variance is estimated separately from an earlier campaign on the same evaluation suite in which five conditions were each trained under two instruction-tuning seeds, giving fifty condition and task pairs.

L PROOF OF PROPOSITION 1

Proof. The samples are independent, so the index of the first accepted sample is independent of whether that sample is correct. Acceptance occurs at least once with probability $1 - (1 - a - b)^k$. Conditioned on acceptance, the accepted sample is drawn from the acceptance event, in which correctness has probability $a/(a + b)$. Substituting $b = 0$ gives $1 - (1 - a)^k$. $\square$

M EXTENDED TESTBED DESCRIPTION

Each example begins with a latent directed acyclic proof. The generator emits a prompt containing ground facts, rules, a query and distractor branches, then traverses the supporting path to produce the target answer. A rendering function writes that path in one of two trace languages. The FORMAL rendering declares constants and predicates and applies named inference rules to numbered lines. The NATURAL rendering expresses the same substitutions and conclusions, line for line, in controlled sentences. Both targets end with the same answer field, and the prompt is natural language in both conditions.

BRANCHPROOF tests composition across a chain of Horn-style inferences in which every step offers four locally plausible branches, exactly one of which is derivable from the current state. Every layer introduces a fresh constant. The generator computes the full Horn closure of each example and accepts it only when exactly one candidate answer is derivable, which removes the possibility that a model is rewarded for guessing among several defensible answers. ATTRCON propagates values among object attributes under generated constraints. Its vocabulary and proof construction are disjoint from BRANCHPROOF, so it provides an independent test of the same hypothesis. Appendix G gives complete paired targets, Appendices E and F specify the grammar, the generator and the validators, and Appendix A documents the answer-uniqueness audit of the final generator.

Generation parameters are held fixed across every controlled experiment reported below. The branching factor is 4, the distractor ratio is 0.5, and the schema variant is the one denoted `hard_fsa_schema` in the release. Depths are sampled round-robin so that each depth in the training range contributes an equal number of examples. Seeds are recorded per corpus and are disjoint across roles, so that no corpus used for training shares a seed with a corpus used for evaluation or for sampling.